

Universidad Autónoma de Entre Ríos

Facultad de Ciencia y Tecnología

Ingeniería de Software II

TP N°3 - Patrones de Creación

Integrantes:

- Baez, Brenda Agustina
- Blanc, Maitén Guadalupe
- Brogna, Aarón

1. Provea una clase que dado un número entero cualquiera retorne el factorial del mismo, debe asegurarse que todas las clases que lo invoquen utilicen la misma instancia de clase.

Para resolver este ejercicio, se debe usar el patrón creacional **Singleton**, el cual permite asegurarnos de que una clase tenga una única instancia y, a la vez, proporciona un punto de acceso global a la misma.

El código que resuelve esto se encuentra en el siguiente link:

https://github.com/MaitenBlanc/UADER_IS2_BLANC/blob/TP3/TP3/src/factorial_singleton.py

Respuesta al ejecutarlo:

```
(IS2) maiten@maiten-H81:~/Escritorio/UADER/3-año/Ingeniería de Software II/Trabajos Paracticos/VSCode/TP3/src$ python3 factorial_singleton.py
¿Son la misma instancia? True
Factorial de 5: 120
Factorial de 8: 40320
Factorial de 3: 6
```

2. Elabore una clase para el cálculo del valor de impuestos a ser utilizado por todas las clases que necesiten realizarlo. El cálculo de impuestos simplificado deberá recibir un valor de importe base imponible y deberá retornar la suma del cálculo de IVA (21%), IIBB (5%) y Contribuciones municipales (1,2%) sobre esa base imponible.

Para este ejercicio utilicé el patrón **Singleton** ya que el enunciado pide que todas las clases usen la misma instancia para calcular impuestos.

El código que resuelve esto se encuentra en el siguiente link:

https://github.com/baezbrenda05/UADER_IS2_Baez/blob/main/TP3/EJ_2.py

Respuesta al ejecutarlo:

```
bbaez@weiszbzba MINGW64 ~/UADER_IS2_Baez-1 (main)
$ C:/Users/bbaez/AppData/Local/Python/pythoncore-3.14-64/python.exe c:/Users/bbaez/UADER_IS2_Baez-1/TP3/EJ_2.py
¿Son la misma instancia? True
Impuestos sobre 10000:
  IVA (21%): 2100.0
  IIBB (5%): 500.0
  Contribuciones municipales (1.2%): 120.0
Total impuestos: 2720.0
```

3. Genere una clase donde se instancie una comida rápida "hamburguesa" que pueda ser entregada en mostrador, retirada por el cliente o enviada por delivery. A los efectos prácticos bastará que la clase imprima el método de entrega.

Para este ejercicio utilicé el patrón **Factory** ya que necesitaba crear distintos tipos de hamburguesa según el método de entrega sin que el código cliente supiera qué clase concreta se estaba creando.

El código que resuelve esto se encuentra en el siguiente link:

https://github.com/baezbrenda05/UADER_IS2_Baez/blob/main/TP3/EJ_3.py

Respuesta al ejecutarlo:

```
bbaez@weiszba MINGW64 ~/UADER_IS2_Baez-1 (main)
$ C:/Users/bbaez/AppData/Local/Python/pythoncore-3.14-64/python.exe c:/Users/bbaez/UADER_IS2_Baez-1/TP3/EJ_3.py
Hamburguesa entregada en mostrador - Tiempo estimado: 5 minutos
Hamburguesa retirada por el cliente - Tiempo estimado: 10 minutos
Hamburguesa enviada por delivery - Tiempo estimado: 30 minutos
```

4. Implemente una clase “factura” que tenga un importe correspondiente al total de la factura pero de acuerdo a la condición impositiva del cliente (IVA Responsable, IVA No Inscripto, IVA Exento) genere facturas que indiquen tal condición.
Para este ejercicio utilicé el patrón **Factory** para generar distintos tipos de factura según la condición impositiva del cliente.

El código que resuelve esto se encuentra en el siguiente link:

https://github.com/baezbrenda05/UADER_IS2_Baez/blob/main/TP3/EJ_4.py

Respuesta al ejecutarlo:

```
bbaez@weiszba MINGW64 ~/UADER_IS2_Baez-1 (main)
$ C:/Users/bbaez/AppData/Local/Python/pythoncore-3.14-64/python.exe c:/Users/bbaez/UADER_IS2_Baez-1/TP3/EJ_4.py
Factura A - IVA Responsable - Importe: 500000 - Descuento 10%: 50000.0 - Total: 450000.0
Factura B - IVA No Inscripto - Importe: 350000 - Descuento 5%: 17500.0 - Total: 332500.0
Factura C - IVA Exento - Importe: 750000 - Sin descuento - Total: 750000
```

5. Extienda el ejemplo visto en el taller en clase de forma que se pueda utilizar para construir aviones en lugar de vehículos. Para simplificar suponga que un avión tiene un “body”, 2 turbinas, 2 alas y un tren de aterrizaje.
Para resolver este ejercicio, se debe usar el patrón creacional **Builder**, el cual permite construir objetos complejos paso a paso. Es decir, permite producir distintos tipos y representaciones de un objeto empleando el mismo código de construcción.

El código que resuelve esto se encuentra en el siguiente link:

https://github.com/MaitenBlanc/UADER_IS2_BLANC/blob/TP3/TP3/src/avion_builder.py

Respuesta al ejecutarlo:

```
(IS2) maiten@maiten-H81:~/Escritorio/UADER/3-anio/Ingenieria de Software II/Trabajos Paracticos/VSCode/TP3/src$ python3 avion_builder.py
Avión construido con:
- 1 Body
- 2 Turbinas
- 2 Alas
- 1 Tren de aterrizaje
```

6. Dado una clase que implemente el patrón “prototipo” verifique que una clase generada a partir de ella permite por su parte obtener también copias de si misma.

Para realizar el ejercicio se modela un sistema de notificaciones de una aplicación (por ejemplo, WhatsApp), donde múltiples notificaciones comparten una misma configuración base (icono, sonido, app) pero difieren en datos dinámicos como el remitente y el mensaje.

El patrón Prototype se implementa mediante la clase Notificación, que define el

método clone(). Además, se verifica que los objetos clonados también pueden clonarse entre sí, demostrando que cualquier instancia puede actuar como prototipo.

El código que resuelve esto se encuentra en el siguiente link:

https://github.com/aaronbroгна04/UADER_IS2_Brogna/blob/main/TP3/ejercicio6.py

Respuesta al ejecutarlo:

```
(IS2) aaron@aaron-M15:~/Facultad/Ingenieria de Software II/Practica/Trabajos_practicos/TP3$ python3 ejercicio6.py
=== Nuevo mensaje de Juan ===
(Whatsapp)Juan: Hola. Como estas?
(Whatsapp)Juan: Me pasas lo que hicieron hoy?

=== Nuevo mensaje de Maria ===
(Whatsapp)Maria: Te mandé el archivo ayer
```

7. Imagine una situación donde pueda ser de utilidad el patrón “abstract factory”.

En este caso se implementó el patrón Abstract Factory para modelar distintos tipos de menús de comida. La fábrica abstracta FabricaMenu define la creación de una familia de productos relacionados (PlatoPrincipal y Postre), mientras que las fábricas concretas MenuVegano y MenuTradicional generan combinaciones consistentes para cada tipo de menú. Los productos concretos representan las opciones específicas de cada familia, y la clase Restaurante actúa como cliente utilizando únicamente las abstracciones, sin depender de clases concretas. De esta forma, el patrón permite mantener consistencia entre productos y facilitar la incorporación de nuevos tipos de menús.

El código que resuelve esto se encuentra en el siguiente link:

https://github.com/aaronbroгна04/UADER_IS2_Brogna/blob/main/TP3/ejercicio7.py

Respuesta al ejecutarlo:

```
(IS2) aaron@aaron-M15:~/Facultad/Ingenieria de Software II/Practica/TrabajosPracticos/TP3$ python3 ejercicio7.py
=== Menú vegano ===
Sirviendo milanesa de berenjena
Sirviendo mousse vegano

=== Menú tradicional ===
Sirviendo milanesa de carne
Sirviendo mousse tradicional
```